Largest square rectangle with all 1’s

Problem Statement

Given a matrix of dimensions mxn having all entries as 1 or 0, find out the size of maximum size square sub-matrix with all 1s.

Example

Consider the matrix: Some square sub matrices with all 1s:

0	1	1	0	1	1
1	1	0	1	1	1
0	1	1	1	0	0
1	1	1	1	0	0
1	1	1	1	1	0
0	1	1	1	0	1

Maximum size square sub matrix with all 1s has size = 3

IDeserve ©

ID

Algorithm / Insights

1. Create an empty table of size mxn, defined as
   table[i][j] = Size of maximum square sub matrix with all 1s with matrix[i][j] as bottom right element

0	1	1	0	1	1
1	1	0	1	1	1
0	1	1	1	0	0
1	1	1	1	0	0
1	1	1	1	1	0
0	1	1	1	0	1

Maximum square sub matrix with all 1s with matrix[3][2] as bottom right element

table[3][2] = 2

IDeserve ©

ID

Algorithm / Insights

1. Create an empty table of size mxn, defined as
   table[i][j] = Size of maximum square sub matrix with all 1s with matrix[i][j] as bottom right element

2. Set values in table based on the conditions:
   a. if i = 0 or j = 0, table[i][j] = matrix[i][j]

0	1	1	0
1	1	0	1
0	1	1	1
1	1	1	1

When i = 0 or j = 0 i.e. in first row or first column :
If matrix[i][j] = 0, then the maximum size of sub matrix with all 1s is 0.

IDeserve ©

ID

Algorithm / Insights

1. Create an empty table of size mxn, defined as
   table[i][j] = Size of maximum square sub matrix with all 1s with matrix[i][j] as bottom right element

2. Set values in table based on the conditions:
   a. if i = 0 or j = 0, table[i][j] = matrix[i][j]

0	1	1	0
1	1	0	1
0	1	1	1
1	1	1	1

When i = 0 or j = 0 i.e. in first row or first column :
If matrix[i][j] = 0, then the maximum size of sub matrix with all 1s is 0.
If matrix[i][j] = 1, then the element itself is largest square sub matrix with all 1s, so size is 1.

IDeserve ©

ID

Algorithm / Insights

1. Create an empty table of size mxn, defined as table[i][j] = Size of maximum square sub matrix with all 1s with matrix[i][j] as bottom right element

2. Set values in table based on the conditions:
   a. if i = 0 or j = 0, table[i][j] = matrix[i][j]
   b. else if matrix[i][j] = 0 then table[i][j] = 0

   Why? Because there cannot be a submatrix that has all 1's with this 0 cell as the bottom right cell

IDeserve ©

Algorithm / Insights

1. Create an empty table of size mxn, defined as
   table[i][j] = Size of maximum square sub matrix with all
   1s with matrix[i][j] as bottom right element

2. Set values in table based on the conditions:
   a. if i = 0 or j = 0, table[i][j] = matrix[i][j]
   b. else if matrix[i][j] = 0 then table[i][j] = 0
   c. else table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1

IDeserve ©

Algorithm / Insights

1. Create an empty table of size mxn, defined as
   table[i][j] = Size of maximum square sub matrix with all 1s with matrix[i][j] as bottom right element

2. Set values in table based on the conditions:
   a. if i = 0 or j = 0, table[i][j] = matrix[i][j]
   b. else if matrix[i][j] = 0 then table[i][j] = 0
   c. else table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1

During step 2, maintain largest element added in the table and finally return it.

IDeserve ©

Algorithm / Insights

Why do we set:

table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1 ?

1	1	1	1	0
0	1	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	1	1	0

1	1	1	1	0
0	1	2	2	1
0	1	2	3	2
1	0	1	2	
0				

To fill this entry

IDeserve ©

ID

Algorithm / Insights

Why do we set:

table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1 ?

1	1	1	1	0
0	1	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	1	1	0

1	1	1	1	0
0	1	2	2	1
0	1	2	3	2
1	0	1	2	
0				

There is a square sub matrix of size 2 having this cell as bottom right corner

IDeserve ©

Algorithm / Insights

Why do we set:

table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1 ?

1	1	1	1	0
0	1	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	1	1	0

1	1	1	1	0
0	1	2	2	1
0	1	2	3	2
1	0	1	2	
0				

There is a square sub matrix of size 2 having this cell as bottom right corner

IDeserve ©

Algorithm / Insights

Why do we set:

table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1 ?

1	1	1	1	0
0	1	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	1	1	0

1	1	1	1	0
0	1	2	2	1
0	1	2	3	2
1	0	1	2	
0				

There is a square sub matrix of size 2 having this cell as bottom right corner

IDeserve ©

Algorithm / Insights

Why do we set:

table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1 ?

1	1	1	1	0
0	1	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	1	1	0

1	1	1	1	0
0	1	2	2	1
0	1	2	3	2
1	0	1	2	
0				

There is a square sub matrix of size 3 having this cell as bottom right corner

IDeserve ©

ID

Algorithm / Insights

Why do we set:

table[i][j] = Min(matrix[i-1][j], matrix[i-1][j-1], matrix[i][j-1]) + 1 ?

1	1	1	1	0
0	1	1	1	1
0	1	1	1	1
1	0	1	1	1
0	1	1	1	0

1	1	1	1	0
0	1	2	2	1
0	1	2	3	2
1	0	1	2	3
0				

Clearly we can see the max size sq. sub matrix will all 1's including this cell is of size 3.

If we do not take the minimum size then we will be including a 0 in the submatrix

IDeserve ©

ID

Example

Matrix

1	1	0	1	1
0	1	1	1	1
0	0	1	1	1
1	1	1	1	1
0	1	1	1	0

Table

1	1	0	1	1
0	1	1	1	2
0	0			
1				
0				

Min + 1

IDeserve ©

ID

Example

Matrix

1	1	0	1	1
0	1	1	1	1
0	0	1	1	1
1	1	1	1	1
0	1	1	1	0

Table

1	1	0	1	1
0	1	1	1	2
0	0	1	2	2
1	1	1	2	3
0	1	2	2	0

Largest element in table = 3

Hence return 3

IDeserve ©

ID

https://www.youtube.com/watch?v=FO7VXDfS8Gk